MongoDB Database Schema for Smart TRIP

This document contains the recommended schema definitions for the core collections of the Smart TRIP project: `Users`, `Vendors`, `Inventory`, and `Bookings`.

1. `User` Collection
Stores information for all individuals who can log in, distinguished by their role.

```javascript
const userSchema = new mongoose.Schema({
  name: {
    type: String,
    required: true
  },
  email: {
    type: String,
    required: true,
    unique: true
  },
  password: {
    type: String,
    required: true
    // Will be stored as a hash
  },
  role: {
    type: String,
    enum: ['user', 'vendor', 'admin'],
    required: true,
    default: 'user'
  }
}, { timestamps: true });
```

2. `Vendor` Collection
Stores business-specific details for users with the 'vendor' role. This allows for a clear separation between login identity and business entity.

```javascript
const vendorSchema = new mongoose.Schema({
  user: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: true
    // Links to the user with role: 'vendor'
  },
  businessName: {
    type: String,
    required: true
  },
  description: {
    type: String
  },
  contactInfo: {
    phone: String,
    address: String
  },
  verificationStatus: {
    type: String,
    enum: ['pending', 'approved', 'rejected'],
    default: 'pending'
  }
}, { timestamps: true });
```

3. `Inventory` Collection
A flexible collection to store all types of services offered by vendors (accommodations, transport, activities). A `type` field is used to differentiate them, allowing the optimization algorithm to query a single collection.

```javascript
const inventorySchema = new mongoose.Schema({
  vendor: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Vendor',
    required: true
  },
  type: {
    type: String,
    enum: ['accommodation', 'transport', 'activity'],
    required: true
  },
  name: {
    type: String,
    required: true
  },
  description: {
    type: String
  },
```

```javascript
  },
  location: {
    // For geospatial queries, e.g., finding activities near a hotel
    type: { type: String, enum: ['Point'], default: 'Point' },
    coordinates: { type: [Number], index: '2dsphere' } // [longitude, latitude]
  },
  basePrice: {
    type: Number,
    required: true
  },
  priceUnit: {
    type: String, // e.g., 'per_night', 'per_person', 'per_trip'
    required: true
  },
  // --- Fields specific to 'accommodation' type ---
  accommodationDetails: {
    category: String, // e.g., 'Hotel', 'Guest House'
    amenities: [String], // e.g., 'Pool', 'Wi-Fi'
    mealPlans: [String] // e.g., 'Room Only', 'Full Board'
  },
  // --- Fields specific to 'transport' type ---
  transportDetails: {
    category: String, // e.g., 'Car', 'Van'
    capacity: Number // max passengers
  },
```

```
  // --- Fields specific to 'activity' type ---
  activityDetails: {
    category: String // e.g., 'Cultural', 'Adventure', 'Nature'
  }
}, { timestamps: true });
```

4. `Booking` Collection
Stores all booking requests and their current status, linking the user with the inventory items they have selected.

```
const bookingSchema = new mongoose.Schema({
  user: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: true
    // The user who made the booking
  },
  items: [{
    inventory: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'Inventory'
    },
    priceAtBooking: {
      type: Number,
      required: true
    }
  }],
  totalCost: {
    type: Number,
    required: true
  },
  tripDates: {
    startDate: { type: Date, required: true },
    endDate: { type: Date, required: true }
  },
  pax: { // Number of people
    adults: { type: Number, default: 1 },
    children: { type: Number, default: 0 }
  },
```

```
  status: {
    type: String,
    enum: ['pending', 'confirmed', 'rejected', 'cancelled'],
    default: 'pending',
    required: true
  }
}, { timestamps: true });
```